

Does a tabular foundation model need preprocessing? TabFM against conventional models on multi-site heart disease data

Course: CSE437 Data Science · Section 5 · Summer 2026 semester

Repository: <https://github.com/RayzerGT/CSE437-TabFM-on-HeartDisease-Dataset-Group3>

Date: 3rd September 2026

Group 3

Name	Student ID
Razeen Hassan	23101133

Summary

Standard Machine Learning has been long dependent on legacy learning algorithms such as XGBoost, Random Forest and Logistic Regression, which provide good performance but require a significant amount of preprocessing before being able to do so. This preprocessing stage of machine learning paradigms takes up a significant amount of time and requires in depth understanding of the datasets themselves for optimal transformation. TabFM, a foundation model, released in June of 2026, has completely changed the pre-existing paradigms. By offering a “Zero Shot” solution, TabFM is capable of processing large amounts of messy data without requiring any preprocessing and producing outputs in one step.

This study compares 5 head-to-head models using the UCI Heart Disease dataset, a dataset notoriously known for being messy and requiring thorough preprocessing. TabFM is run on both a preprocessing and a raw version of the dataset and 3 baseline legacy models are used for comparison. TabFM on raw data was best on every metric, reaching 0.953 ROC-AUC against 0.941 for the strongest baseline and missing only 9 diseased patients where the baselines missed 14 or 15. The most important finding is that cleaning the data made TabFM worse: the preprocessed arm scored 0.939, below both the raw arm and logistic regression. On this dataset, preprocessing cost accuracy rather than buying it.

1. Problem and Dataset

1.1 Problem statement

Over the years machine learning algorithms such as XGBoost, Decision Trees (All variants) and Logistic Regression have been at the forefront of computing and analyzing tabular data. Although reliable and proven to work in most scenarios, the amount of time required to preprocess large datasets has proven to be cumbersome. Copious amounts of time have to be poured into figuring out ways to transform the dataset so that it reach its ideal state: a state capable of training models fast and efficiently.

The modern-day solution to this recurrent problem has been found through the invention of TabFM. Released by Google Research in June 2026, the foundation model reports beating established tabular algorithms on the TabArena benchmark suite without requiring prior modifications on datasets, providing a direct zero-shot solution to model training.

This project asks whether the advantage survives that setting, and whether “no feature engineering required” holds when the raw data is genuinely distorted. We fit five models to the UCI Heart Disease

dataset: TabFM on minimally processed data, TabFM on conventionally cleaned data, and three conventional baseline models: XGBoost, Logistic regression and Decision Tree.

1.2 Dataset

Source: UCI Machine Learning Repository - <https://archive.ics.uci.edu/dataset/45/heart+disease>

The dataset includes a collection of features including demographics, symptoms, resting measurements and exercise stress-test results recorded beforehand, and the label is the angiographic result (Whether the patient has heart disease or not). It is derived from an angiographic study reported by Detrano et al. (1989), donated to UCI in 1988. The raw dataset was downloaded as four independently collected hospital databases, which we merged by concatenation, adding an extra column representing the source hospital.

Size: 920 rows $\times$ 13 features with an extra column the label.

Time period: Collected during the 1980s, donated 1988.

Licence: Creative Commons Attribution 4.0 International. Patient identifiers were replaced with dummy values before release.

1.3 Target variable

- Column name: target
- Type: Discrete Binary

Note: The data is present in multiclass form but for simplicity the project has been set up for binary classification. As such label value 0 is considered as negative while all other classes (1-4) are considered positive. This has been done for simplicity as the main goal of the study is to test and compare the state-of-the-art foundation model TabFM to legacy machine learning methods.

Raw class distribution:

num	0	1	2	3	4
Frequency	411	265	109	107	28

This strategy also mitigates the class imbalance problem that would arise if it was treated as a multiclass problem.

Simplified Class distribution:

1. [0] – Negative Class (44.6 %)
2. [1-4] – Positive class (55.4%)

1.4 Research Questions

1. Does TabFM predict heart disease more accurately than logistic regression, random forest, and XGBoost?
2. Does cleaning the data change how well TabFM performs?
3. Which model holds up best when tested on a hospital it has not seen before?

2. Data Handling and Preprocessing

2.1 Data quality audit

Data quality audit
UCI Heart Disease, four hospital databases · 920 rows x 13 feature columns · 299 complete rows (32.5%) · 2 duplicate rows

Column	Type	Distinct	Missing	Missing rate	Impossible values	Unexpected levels
age	continuous	50	0	0.0%	—	none
sex	binary	2	0	0.0%	—	none
cp	categorical	4	0	0.0%	—	none
trestbps	continuous	61	59	6.4%	1 (value 0)	none
chol	continuous	217	30	3.3%	172 (value 0)	none
fbs	binary	2	90	9.8%	—	none
restecg	categorical	3	2	0.2%	—	none
thalach	continuous	119	55	6.0%	—	none
exang	binary	2	55	6.0%	—	none
oldpeak	continuous	53	62	6.7%	—	none
slope	ordinal	3	309	33.6%	—	none
ca	ordinal	4	611	66.4%	—	none
thal	categorical	3	486	52.8%	—	none

Duplicate rows: 2 exact duplicates across all 14 columns (Hungarian ids 404/405, VA ids 859/907). Retained, not dropped.
Impossible values: serum cholesterol and resting blood pressure recorded as 0. All 123 Switzerland records and 49 VA records affected.
Unexpected categories: none. Every coded column uses only the levels defined in the dataset documentation.
Missing values are marked "?" in the source files. Missingness is structured by hospital, not random — see Figure: missing values by hospital.

Missing values: Only age, sex and cp are complete. ca is absent for 66.4% of records, thal for 52.8%, slope for 33.6%. Just 299 of 920 records (32.5%) are complete cases. Missing entries are marked “?” in the source files.

- **Duplicate rows:** Two duplicate rows found (Same record from two different hospitals)
- **Inconsistent categories:** None
- **Impossible values:** 172 cholesterol readings and one blood pressure reading are recorded as exactly 0, which is logically impossible.

2. Data Handling and Preprocessing

2.2 Missing values

Column	Missing	% of 918	Strategy
ca	611	66.4%	Column dropped
thal	486	52.8%	Column dropped
slope	307	33.4%	Mode imputation
fbs	90	9.8%	Mode imputation
oldpeak	62	6.8%	Median imputation
trestbps	59	6.4%	Median imputation
thalach	55	6.0%	Median imputation
exang	55	6.0%	Mode imputation
chol	29	3.2%	Median imputation
restecg	2	0.2%	Mode imputation
age, sex, cp	0	0%	None needed

The ca and thal columns were missing most of their records and as such they were dropped. Imputation on those columns might increase the noise present within them so the risk was not taken. Again, as the

purpose of the experiment is to evaluate TabFM performance excessive preprocessing was not carried out. For all other missing data records, continuous variables were imputed with their median and discrete features were imputed with their mode.

2.3 Outliers

Detection mechanism: Tukey's rule ($1.5 \times \text{IQR}$)

Flagged, over all 918 records:

Feature	Flagged	Fence	Note
chol	185	[42.6, 401.6]	Includes all 172 recorded zeros
trestbps	28	[90.0, 170.0]	Includes the one recorded zero
oldpeak	16	[-2.2, 3.8]	
thalach	2	[66.4, 209.4]	
age	0	[27.5, 79.5]	

Action: Any record flagged on any column was removed: 217 records dropped, leaving 701.

2.4 Transformation and scaling

Encoding: One hot encoding used for all categorical features with 1 column dropped from each to reduce redundancy.

Column	Levels	Dummies created
sex	0 female, 1 male	sex_1
cp	1, 2, 3, 4	cp_2, cp_3, cp_4
fbs	0, 1	fbs_1
restecg	0, 1, 2	restecg_1, restecg_2
exang	0, 1	exang_1
slope	1, 2, 3	slope_2, slope_3

Six columns become ten one-hot encoded dummies, increasing feature columns to 15.

Transformations:

- No transformations: Oldpeak (Minimal Variance)
- Standardization: age, trestbps, chol, thalach (High Variance). Standardization was used instead of min-max scaling because standardization is less sensitive to extreme values. Although outliers were removed, standardization seemed appropriate for this situation due to the wide range of values per feature.

Filtering: Experiment setup to remove correlating pairs with values above 0.90. None of the columns dropped as the largest figure came out to be 0.53. (Found through EDA)

2.5 Before and after

Stage	Rows	Cols
Loaded, merged, target derived	920	13
Dropped columns >50% missing	920	11
Dropped duplicate rows	918	11
Imputed remaining missing values	918	11
Dropped $1.5 \times$ IQR outliers	701	11
One-hot encoded coded columns	701	15
Standardized continuous columns	701	15

2.6 Leakage control

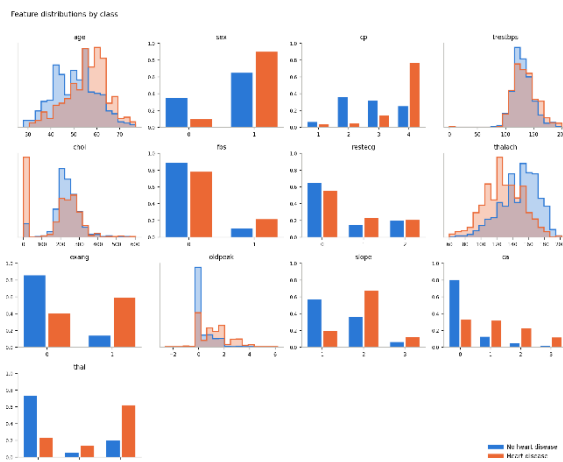
Leakages guarded. The training and test splits are done before all preprocessing is applied. After that preprocessing methods are applied independently to each set to not leak latent information from the training split to the test split. Cross validation is done using the training split only. As such the test split stays completely uninfluenced by the training split.

3. Statistical Analysis

3.1 Descriptive statistics

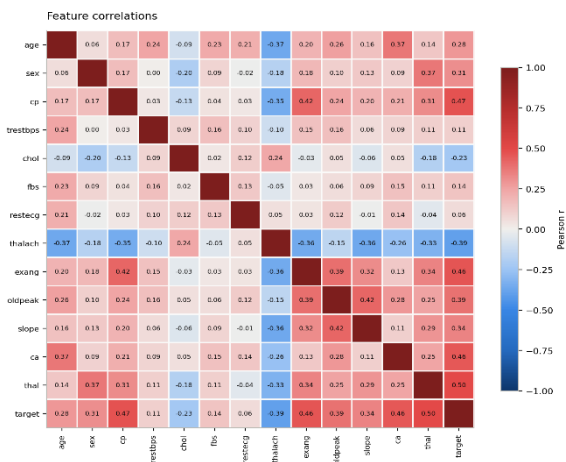
Feature	n	Mean	SD	Median	IQR	Skew
age	920	53.5	9.4	54	13.0	-0.20
trestbps	861	132.1	19.1	130	20.0	0.21
chol	890	199.1	110.8	223	93.0	-0.61
thalach	865	137.6	25.9	140	37.0	-0.21
oldpeak	858	0.88	1.09	0.5	1.5	1.04

3.2 Relationships



Note:

- The chol, thalach, ca and exang features do not follow a completely normal distribution but seem to be close to one. As such using standardization seems ideal for those features.
- The cp, ca features seem to be highly correlated with the label within the distribution plots.
- The thalach feature seems to have high negative correlation with the target label.
- As observed from the distribution plots above, the target label has a high positive correlation with ca and cp features. Unfortunately, the cp feature had to be dropped due to the lack of data points



3.3 What the data says so far

1. The cp and thal feature seem to show high predictive potential but must be dropped due to a severe lack of records. (Did not want to risk introducing noise).
2. The features chol, thalach and trestbps have wide value ranges and as such must be scaled.
3. The chol feature has heavy negative skew.
4. The oldpeak feature has low standard deviation, so scaling is not required.

4. Feature Engineering

4.1 Derived features

The label column of the raw dataset came as a multiclass classification problem with 5 distinct classes with high disproportionality between classes. To avoid overcomplication and prevent severe imbalances between target class values, the problem was transformed into a binary classification problem with class 0 as the negative outcome and classes 1,2,3,4 as the positive outcome.

4.2 Dimensionality reduction

- Done through dropping the extra redundant columns after one-hot encoding
- Did not apply PCA for the sake of simplicity. The scope of the study is to assess TabFM's performance and as such did not want to deviate from the primary goal.

4.3 Feature selection

Selection was performed by three threshold-based filters applied in sequence.

Filter	Threshold	Result
Missingness	Drop column if >50% missing	ca (66.4%), thal (52.8%) dropped
Variance	Drop if ≤ 1 distinct value	None dropped
Redundancy	Drop one of any pair with $ r > 0.90$	None dropped (strongest pair 0.53)

4.4 Final feature set

Fifteen columns: five continuous and ten discrete binary.

Type	Columns	Source
Continuous, standardised	age, trestbps, chol, thalach, oldpeak	Original measurements
Binary dummies	sex_1	from sex
	cp_2, cp_3, cp_4	from cp, reference cp_1
	fbs_1	from fbs
	restecg_1, restecg_2	from restecg, reference restecg_0
	exang_1	from exang
	slope_2, slope_3	from slope, reference slope_1

5. Modeling and Validation

5.1 Validation strategy

A stratified 80/20 holdout: 141 test patients, stratified jointly on hospital and outcome. Seed 42, frozen to disk before any model is fitted. Hyperparameter search runs as 5-fold stratified cross-validation inside the training split only; the test set is scored once per model at the end. To answer the question of performance across unseen hospital data, separate grouped splits were made with a random hospital being left out in each grouping. As such the experiment has three completely different group sets: Raw data split for TabFM(Raw), processed data split for TabFM(pre-processed) and the baseline models and the LOHO(Leave-One-Hospital-Out) split.

5.2 Baselines

The legacy models XGBoost, random forest and Logistic Regression were used as a baseline to assess the performance of TabFM foundation model.

5.3 Model families

Family	Why it suits this problem
Foundation model (TabFM, 1–2)	No tuning, no feature engineering, no preprocessing: The performance assessment
Linear (logistic regression, 3)	Baseline against the foundation models
Bagged trees (random forest, 4)	Baseline against the foundation models
Boosted trees (XGBoost, 5)	The long-running state of the art model before Foundation models became ubiquitous

5.4 Metrics

Primary metric: ROC-AUC. It is threshold-free and insensitive to class balance. As such it gives an overall performance measure of the models. Reported alongside it: PR-AUC; precision, recall and F1 at the 0.5 threshold. Confusion matrices are provided for visualization.

6. Hyperparameter Tuning

6.1 Search space

Model	Hyperparameter	Values searched	Candidates
Logistic regression	C	0.01, 0.1, 1, 10, 100	5
Random forest	n_estimators	300, 600	12
	max_depth	None, 4, 8	
	min_samples_leaf	1, 3	
XGBoost	max_depth	2, 3, 5	24
	learning_rate	0.03, 0.1	
	n_estimators	200, 500	
	subsample	0.8, 1.0	
TabFM	N/A		

6.2 Method

Exhaustive grid search over 41 candidates, each evaluated by 5-fold stratified cross-validation and scored by ROC-AUC. The best configuration is refitted on the full training split and scored once on the test set.

6.3 Results

Model	Best configuration	CV ROC-AUC	SD
Logistic regression	C = 1	0.872	0.024
Random forest	max_depth 8, min_samples_leaf 1, n_estimators 300	0.872	0.027
XGBoost	max_depth 2, learning_rate 0.03, n_estimators 200, subsample 0.8	0.870	0.018

7. Results, Visualization and Error Analysis

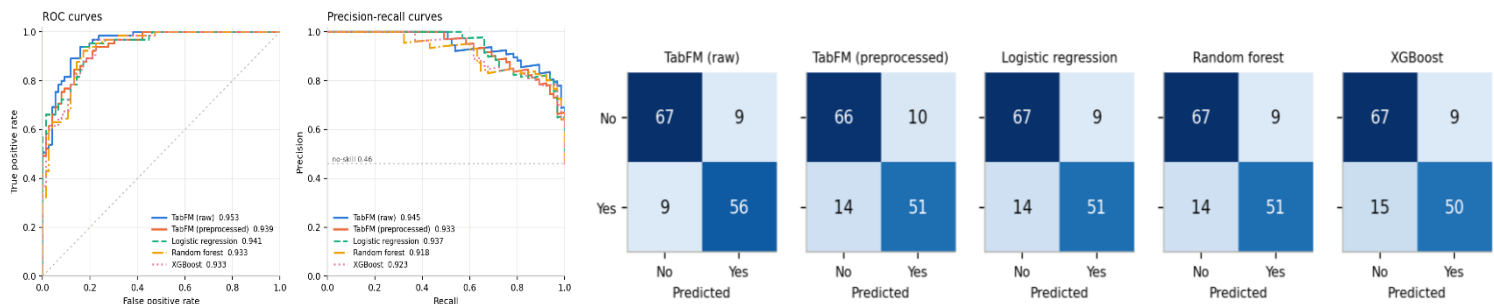
7.1 Test set performance

Scored once, on the 141 held-out records, after all selection was complete.

Model	ROC-AUC	PR-AUC	Accuracy	F1	MCC
TabFM (raw)	0.953	0.944	0.872	0.862	0.743
TabFM (preprocessed)	0.939	0.933	0.830	0.810	0.657
Logistic regression	0.941	0.937	0.837	0.816	0.672
Random forest	0.933	0.918	0.837	0.816	0.672
XGBoost	0.933	0.923	0.830	0.806	0.658

Clearly TabFM is the superior model. Addressing the research questions, we can see that the foundation model works well especially on raw unprocessed datasets.

7.2 Visualization



Many more plots have been created but due to page limit constraints, most of them have been kept outside the report. All of them can be found in the /figures directory of the GitHub repository.

7.3 Error analysis

From the confusion matrix it can be observed that there are very minor differences between all models. The TabFM model with raw data seems to better deal with positive data instances with only 9 misclassifications on positive data. All other models have a misclassification range of 14-15 showcasing TabFM's superior capabilities.

On the ROC-AUC curve there is a major disparity between TabFM (Raw) and XGBoost its closest state of the art competitor. The Precision-Recall curves show interesting results with the TabFM(Raw) being beaten by logistic regression at precision with lower recall but the foundation model quickly closes the gap and surpasses all model at higher recall measures.

7.4 Answers to the three questions

Question 1 — Does TabFM predict heart disease more accurately than logistic regression, random forest and XGBoost? Yes, on this test set, and on every metric. TabFM on raw data reaches 0.953 ROC-AUC against 0.941, 0.933 and 0.933, and 0.743 MCC against 0.672, 0.672 and 0.658. It also misses 9 diseased patients where the baselines miss 14 or 15.

Question 2 — Does cleaning the data change how well TabFM performs? Yes, and it made it worse. The raw arm scores 0.953 and the preprocessed arm 0.939; the gap on MCC is larger, 0.743 against 0.657.

Question 3 — Which model holds up best on an unseen hospital? Not much of a performance disparity was observed between models on unseen hospital splits. Mean ROC-AUC across the three held-out hospitals (figures/loho_transfer.png) is 0.844 TabFM raw, 0.842 logistic regression, 0.839 TabFM preprocessed and random forest, 0.833 XGBoost with a maximum to minimum performance disparity of 0.011.

8. Limitations and Next Steps

A hospital was completely left out due to inconsistent data. The $1.5 \times \text{IQR}$ rule removed all 123 Switzerland records because their cholesterol is recorded as zero. The preprocessing pipeline could be improved, which might influence results. A thorough and granular preprocessing pipeline can be set up to better gauge the performance disparity between raw and preprocessed TabFM training.

The sample is small and narrow. At 141 test records one or two predictions move accuracy by over a percentage point, and no confidence intervals were computed.

TabFM is also the most expensive and least inspectable model here: 138 seconds against 1.3 for XGBoost, a multi-gigabyte dependency, and no feature importances. Whether a 0.012 ROC-AUC advantage justifies that is a deployment question this study does not answer.

9. Contributions

Member	Student ID	Contribution
Razeen Hassan	23101133	The Whole Project

References

1. Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1988). Heart Disease [Dataset]. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/45/heart+disease> . Licensed CC BY 4.0.
2. Google Research (2026). TabFM: a zero-shot foundation model for tabular data. <https://huggingface.co/google/tabfm-1.0.0-pytorch> · <https://github.com/google-research/tabfm>
3. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830. Used for the baselines, cross-validation, grid search and all metrics.
4. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD '16, 785–794.

AI assistance: Notebook LM was used to simplify the reading process of all the related research papers. Claude was used to debug code and assist in decision making when designing the whole experiment. Claude was also used for assistance when writing the research paper especially for the creation of tables and complex data representation structures.